ROTEIRO DE LABORATÓRIO · PROGRAMAÇÃO BACK-END

Descomplicando o OAuth: autorização segura para APIs

2ª série | 4º bimestre | Semana 25 | Aula 1 | Servidores Web, API e Segurança das Aplicações

Atividade elaborada pelo Prof. Marcos Souza, baseado nos materiais didáticos fornecidos pela Sec. Educ. do Estado de São Paulo para o programa de Educação Profissional Paulista, curso Técnico em Desenvolvimento de Sistemas, na disciplina de Programação Back-End – 2ª série, 4º bimestre, Semana 25, Aula 1, tema "Descomplicando o OAuth: autorização segura para APIs" (unidade Servidores Web, API e Segurança das Aplicações).

A ferramenta de inteligência artificial Claude, desenvolvida pela Anthropic, foi utilizada para otimizar esta atividade.

Modelo: Claude Opus 5.5 (claude-opus-5-5) · gerado em 31/10/2026, às 14:20 (horário de Brasília)

Integrantes do grupo

Nomes: _____

_____

Turma: _____

Contexto do laboratório

Sistema operacional	Windows 10/11 — PowerShell 5.1+ como shell padrão
Ferramentas	Node.js 20 LTS+ (npm 10+), Git 2.40+, Visual Studio Code 1.85+, navegador com DevTools
Dependências npm	Nenhuma — o laboratório usa só módulos nativos do Node (http, crypto, fetch)
Grupos	2 a 4 estudantes por grupo

Duração	2 aulas de 50 min (100 min): 10 estudo · 15 preparação · 40 execução/testes · 25 plano e perguntas · 10 entrega
Objetivo	Implementar e testar, em uma API financeira simulada, o Authorization Code Flow com PKCE, a renovação de tokens e o Client Credentials Flow, e redigir o plano de implementação do grupo.

Cenário

> "Time, recebemos uma solicitação urgente do setor de TI: precisamos revisar nosso processo de autorização para a API financeira, garantindo que o fluxo de OAuth esteja devidamente configurado. Como vocês recomendam que ajustemos nossa implementação com base no que discutimos?"
>
> Situação fictícia produzida pela SEDUC-SP

Arquitetura do laboratório

Papel OAuth	Arquivo	Porta	Responsabilidade
Servidor de Autorização	`src/auth-server.js`	4000	Login, consentimento, emissão e revogação de tokens
Servidor de Recursos (API)	`src/api-financeira.js`	5000	Saldo, transações, transferências e relatórios — valida token e escopo
Cliente (App Finanças)	`src/cliente-web.js`	3000	Aplicação web confidencial que pede autorização em nome do usuário

1. **Cliente** redireciona o navegador ao **provedor** com `state` e `code_challenge`.
2. Usuário faz login **no provedor** e consente os escopos.
3. Provedor devolve um **código** de uso único para a redirect URI registrada.
4. Back-end do cliente troca código + `code_verifier` + segredo por **access** e **refresh token**.
5. Cliente chama a **API** com `Authorization: Bearer`; a API valida token e escopo.

Checklist de recursos

- [x] **Computador com Windows 10/11 e PowerShell 5.1+**
  Confira com `$PSVersionTable.PSVersion`
- [x] **Node.js 20 LTS ou superior (inclui npm 10+)**
  nodejs.org ou `winget install OpenJS.NodeJS.LTS`

- [x] **Git 2.40 ou superior**
  git-scm.com ou `winget install Git.Git`
- [x] **Visual Studio Code 1.85 ou superior**
  code.visualstudio.com ou `winget install Microsoft.VisualStudioCode`
- [x] **Extensão REST Client (humao.rest-client) — opcional**
  Executa o arquivo `testes/requisicoes.http` direto no VS Code
- [x] **Navegador atualizado (Edge, Chrome ou Firefox) com DevTools (F12)**
  Para observar os redirecionamentos na aba Rede
- [x] **Portas 3000, 4000 e 5000 livres**
  Se ocupadas, encerre o outro processo antes do `npm start`
- [x] **Arquivo de partida: `atividade-oauth-api_script.ps1`**
  Script de preparação automática do ambiente
- [x] **Material de referência SEDUC-SP (roteiro SISANO2C2B4S25A1AP)**
  Conceitos de fluxos, configuração e exemplos práticos de OAuth
- [x] **Modelo do plano: `docs/plano-oauth.md`**
  Documento que o grupo completa na etapa de Execução

Procedimento passo a passo

- [x] **Passo 1** **Verificar as ferramentas instaladas** — PowerShell

Abra o PowerShell e confira as versões. Se `npm -v` acusar erro de política de execução, use `npm.cmd -v` ou rode `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

PowerShell — comandos

```
node -v          # esperado v20.x ou superior
npm -v           # esperado 10.x ou superior
git --version    # esperado 2.40 ou superior
code --version   # esperado 1.85 ou superior
```

Resultado esperado: Todas as versões aparecem e atendem ao mínimo.

✔ EXEMPLO DE SUCESSO

```
v22.11.0
10.9.0
git version 2.47.1.windows.1
1.95.3
```

✘ EXEMPLO DE FALHA TÍPICA

```
node : O termo 'node' não é reconhecido como nome de cmdlet...
npm : O arquivo C:\Program Files\nodejs\npm.ps1 não pode ser carregado...
```

Como corrigir: Instale com winget install OpenJS.NodeJS.LTS e reabra o PowerShell; para o erro do npm.ps1 use npm.cmd ou Set-ExecutionPolicy -Scope CurrentUser RemoteSigned.

Passo 2 (Opcional) Preparação automática com o script

PowerShell

O script cria a pasta, grava todos os arquivos, faz o commit inicial, roda os testes, sobe os servidores e abre o navegador — sem perguntas. Cada passo é validado de fato (código de saída, arquivos gerados, respostas HTTP dos servidores, execução do Client Credentials). No fim mostra a tabela passo/ambiente/comando/status, separa o que é automático do que é **ação manual do aluno**, indica a correção de cada falha e salva tudo em `relatorio-preparacao.csv` na pasta do projeto. O script nunca faz login, push ou deploy remoto. Quem usar o script confere os arquivos dos passos 3 a 12 no VS Code e marca esses passos; quem preferir fazer à mão segue os passos 3 a 12.

PowerShell — comandos

```
cd $HOME\Downloads
powershell -ExecutionPolicy Bypass -File .\atividade-oauth-api_script.ps1
# opções: -Destino "C:\outra\pasta"   -SemServidor
```

Resultado esperado: Relatório final sem itens FALHOU (VS Code é opcional).

✔ EXEMPLO DE SUCESSO

```
[OK    ] 1  Node.js >= 20 ... versão 22.11.0
...
Sucesso: 19   Falhas: 0   Avisos (opcionais): 0   Pulados: 0   Manuais: 6
Relatório salvo em ...\relatorio-preparacao.csv
```

✘ EXEMPLO DE FALHA TÍPICA

```
... não pode ser carregado porque a execução de scripts foi desabilitada neste sistema.
```

Como corrigir: Rode exatamente: powershell -ExecutionPolicy Bypass -File .\atividade-oauth-api_script.ps1 (não precisa ser administrador).

Passo 3 Criar a pasta do projeto e o repositório Git

PowerShell + Git

Crie a estrutura `src/`, `testes/` e `docs/`, inicialize o repositório e abra a pasta no VS Code.

PowerShell + Git — comandos

```
$proj = "$HOME\Documents\atividade-oauth-api"
New-Item -ItemType Directory -Force -Path $proj, "$proj\src", "$proj\testes", "$proj\docs" | Out-Null
cd $proj
git init -b main
code .
```

Resultado esperado: O VS Code abre a pasta vazia com as três subpastas.

✔ EXEMPLO DE SUCESSO

```
Initialized empty Git repository in C:/Users/aluno/Documents/atividade-oauth-api/.git/
```

✘ EXEMPLO DE FALHA TÍPICA

```
error: unknown switch `b'  (Git anterior à 2.28)
```

Como corrigir: Atualize o Git (winget upgrade Git.Git) ou use git init seguido de git checkout -b main.

Passo 4 Criar package.json, .gitignore e README.md

VS Code

Na raiz do projeto, crie os três arquivos abaixo. O `package.json` define os atalhos `npm start`, `npm test` e `npm run cc`; não há dependências a instalar.

`package.json` (17 linhas)

package.json

```
{
  "name": "atividade-oauth-api",
  "version": "1.0.0",
  "description": "Laboratório OAuth 2.0 — Authorization Code + PKCE, Refresh Token e Client
Credentials (Programação Back-End)",
  "main": "src/iniciar.js",
  "private": true,
  "scripts": {
    "start": "node src/iniciar.js",
    "test": "node testes/teste-fluxo.js",
    "cc": "node src/client-credentials.js"
  },
  "engines": {
    "node": ">=20"
  },
  "license": "UNLICENSED"
}
```

`.gitignore` (6 linhas)

.gitignore

```
node_modules/
.env
*.log
.servidor.pid
relatorio-preparacao.csv
```

`README.md` (19 linhas)

README.md

```
# Laboratório OAuth 2.0 — API Financeira

Atividade de Programação Back-End (2ª série, 4º bimestre, semana 25, aula 1). Sem dependências
externas: apenas Node.js 20+.

| Papel OAuth | Arquivo | Porta |
|---|---|---|
| Servidor de Autorização | `src/auth-server.js` | 4000 |
| Servidor de Recursos (API) | `src/api-financeira.js` | 5000 |
| Cliente (App Finanças) | `src/cliente-web.js` | 3000 |
```

```powershell
npm test      # testes automáticos de segurança
npm start     # sobe os três servidores -> abra http://localhost:3000 (aluno / senha123)
npm run cc    # (outro terminal) demonstração Client Credentials
$env:ACCESS_TTL=20; npm start   # tokens de 20 s para observar a expiração
```

Plano de implementação do grupo: `docs/plano-oauth.md`.

Resultado esperado: Os três arquivos aparecem no Explorer do VS Code.

✔ EXEMPLO DE SUCESSO

```
Explorer do VS Code: package.json  .gitignore  README.md
```

✘ EXEMPLO DE FALHA TÍPICA

```
npm ERR! JSON.parse Unexpected token } in JSON at position 312
```

Como corrigir: Algum caractere ficou faltando/sobrando ao copiar o package.json: use o botão Copiar e cole o arquivo inteiro.

Passo 5 Registrar a aplicação no provedor (config.js)

VS Code

Este arquivo simula o **registro da aplicação** (Material, item 2.2.1): cada cliente tem `client_id`, segredo, **URLs de redirecionamento exatas** (item 2.2.2), fluxos permitidos e **escopos**. O tempo de vida dos tokens (item 2.2.3) também está aqui.

`src/config.js` (56 linhas)

src/config.js

```
// ============================================================
// config.js — "Registro da aplicação" no provedor OAuth
// Equivale ao que se cadastra no console de um provedor real
// (Google Cloud, Microsoft Entra, Auth0...): client_id, segredo,
// URLs de redirecionamento permitidas, fluxos e escopos.
// ============================================================
const PORTAS = {
  auth: Number(process.env.AUTH_PORT || 4000),      // Servidor de Autorização
  api: Number(process.env.API_PORT || 5000),        // Servidor de Recursos (API financeira)
  cliente: Number(process.env.CLIENT_PORT || 3000), // Aplicação cliente (App Finanças)
};

module.exports = {
```

```
  PORTAS,
  EMISSOR: `http://localhost:${PORTAS.auth}`,
  AUDIENCIA_API: 'api-financeira',
  // Em produção: segredo longo, guardado em variável de ambiente/cofre (NUNCA no Git)
  SEGREDO_ASSINATURA: process.env.OAUTH_SEGREDO || 'troque-este-segredo-em-producao',

  // Tempos de vida (segundos) — tokens de acesso curtos, refresh mais longo
  ACCESS_TOKEN_TTL: Number(process.env.ACCESS_TTL || 120), // 2 min (didático; real: 5 a 15 min)
  REFRESH_TOKEN_TTL: Number(process.env.REFRESH_TTL || 1800), // 30 min
  CODE_TTL: 60, // código de autorização: uso único, vale 60 s

  // Escopos = permissões granulares oferecidas pela API
  ESCOPOS: {
    'saldo:read': 'Consultar saldo da conta',
    'transacoes:read': 'Consultar extrato de transações',
    'transacoes:write': 'Criar transferências (operação sensível)',
    'relatorios:read': 'Gerar relatórios consolidados (servidor-servidor)',
  },

  // Aplicações registradas no provedor
  CLIENTES: [
    {
      client_id: 'app-financas-web',
      client_secret: 'segredo-app-web', // cliente confidencial: o segredo fica só no back-end
      nome: 'App Finanças (Web)',
      redirect_uris: [`http://localhost:${PORTAS.cliente}/callback`], // lista exata (sem curingas)
      grant_types: ['authorization_code', 'refresh_token'],
      escopos_permitidos: ['saldo:read', 'transacoes:read', 'transacoes:write'],
    },
    {
      client_id: 'servico-relatorios',
      client_secret: 'segredo-relatorios',
      nome: 'Serviço de Relatórios (back-end)',
      redirect_uris: [], // sem usuário, sem redirecionamento
      grant_types: ['client_credentials'],
      escopos_permitidos: ['relatorios:read'],
    },
  ],

  // Usuário de demonstração (no provedor, nunca no cliente)
  USUARIOS: [{ id: 'u1', login: 'aluno', senha: 'senha123', nome: 'Aluno(a) Demo' }],
};
```

Resultado esperado: Você identifica os dois clientes registrados e os quatro escopos.

✔ EXEMPLO DE SUCESSO

```
node -e "console.log(require('./src/config').CLIENTES.length)"
2
```

✘ EXEMPLO DE FALHA TÍPICA

```
Error: Cannot find module './src/config'
```

Como corrigir: Confira se o arquivo está em src\config.js (e não em src\config.js.txt) e se o terminal está na raiz do projeto.

Passo 6 Criar os utilitários HTTP e de JWT

VS Code

`util.js` reúne funções de HTTP, geração de valores aleatórios e SHA-256 (usado no PKCE). `jwt.js` assina e valida tokens JWT HS256 com comparação em tempo constante.

`src/util.js` (80 linhas)

src/util.js

```
// ============================================================
// util.js — Funções auxiliares HTTP (somente módulos nativos)
// ============================================================
const crypto = require('node:crypto');


function lerCorpo(req) {
  return new Promise((resolve, reject) => {
    let dados = '';
    req.on('data', (parte) => {
      dados += parte;
      if (dados.length > 1e6) req.destroy(); // limite de 1 MB
    });
    req.on('end', () => {
      try {
        const tipo = req.headers['content-type'] || '';
        if (tipo.includes('application/json')) return resolve(dados ? JSON.parse(dados) : {});
        resolve(Object.fromEntries(new URLSearchParams(dados)));
      } catch (e) {
        reject(e);
      }
    });
    req.on('error', reject);
```

```
  });
}

function json(res, status, obj, extras = {}) {
  res.writeHead(status, { 'Content-Type': 'application/json; charset=utf-8', 'Cache-Control': 'no-
store', ...extras });
  res.end(JSON.stringify(obj, null, 2));
}

function html(res, status, corpo, extras = {}) {
  res.writeHead(status, { 'Content-Type': 'text/html; charset=utf-8', 'Cache-Control': 'no-store',
...extras });
  res.end(corpo);
}

function redirecionar(res, url) {
  res.writeHead(302, { Location: url });
  res.end();
}

function lerCookies(req) {
  return Object.fromEntries(
    (req.headers.cookie || '').split(';').filter((c) => c.includes('=')).map((c) => {
      const i = c.indexOf('=');
      return [c.slice(0, i).trim(), decodeURIComponent(c.slice(i + 1).trim())];
    })
  );
}

const aleatorio = (bytes = 32) => crypto.randomBytes(bytes).toString('base64url');
const sha256b64url = (texto) => crypto.createHash('sha256').update(texto).digest('base64url');

function iguaisSeguro(a, b) {
  const x = Buffer.from(String(a ?? ''));
  const y = Buffer.from(String(b ?? ''));
  return x.length === y.length && crypto.timingSafeEqual(x, y);
}

const esc = (s) =>
  String(s ?? '').replace(/[&<>"']/g, (c) => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"':
'&quot;', "'": '&#39;' }[c]));

function pagina(titulo, corpo, cor = '#2563eb') {
  return `<!doctype html><html lang="pt-BR"><head><meta charset="utf-8">
```

```
<meta name="viewport" content="width=device-width,initial-scale=1"><title>${esc(titulo)}</title>
<style>
 body{font-family:system-ui,Segoe UI,sans-serif;max-width:860px;margin:2rem auto;padding:0
1rem;color:#1f2937;background:#f8fafc}
 .tag{display:inline-block;background:${cor};color:#fff;padding:.2rem .6rem;border-
radius:99px;font-size:.8rem}
 h1{margin:.4rem 0 1rem} h2{font-size:1.05rem;margin-top:1.6rem}
 pre{background:#0f172a;color:#e2e8f0;padding:1rem;border-radius:8px;overflow:auto;font-
size:.85rem}
 .btn,button{display:inline-block;background:${cor};color:#fff;border:0;padding:.55rem
.9rem;border-radius:6px;
   text-decoration:none;font-size:.9rem;cursor:pointer;margin:.2rem .2rem .2rem 0}
 .sec{background:#475569} .perigo{background:#b91c1c}
 form{display:inline} label{display:block;margin:.5rem 0} input{padding:.45rem;border:1px solid
#cbd5e1;border-radius:6px}
 form.bloco{display:block;background:#fff;padding:1rem;border-radius:8px;border:1px solid #e2e8f0}
 .erro{color:#b91c1c;font-weight:600} .dica{font-size:.85rem;color:#475569} ol li{margin:.25rem
0;font-size:.9rem}
</style></head><body>${corpo}</body></html>`;
}

module.exports = { lerCorpo, json, html, redirecionar, lerCookies, aleatorio, sha256b64url,
iguaisSeguro, esc, pagina };
```

`src/jwt.js` (42 linhas)

src/jwt.js

```
// ============================================================
// jwt.js — Assinatura e validação de JWT (HS256) sem bibliotecas
// Em produção prefira bibliotecas consolidadas (ex.: "jose") e
// assinatura assimétrica (RS256/ES256) publicada via JWKS.
// ============================================================
const crypto = require('node:crypto');

const b64 = (obj) => Buffer.from(JSON.stringify(obj)).toString('base64url');
const hmac = (dados, segredo) => crypto.createHmac('sha256',
segredo).update(dados).digest('base64url');

function assinar(payload, segredo) {
  const cabecalho = b64({ alg: 'HS256', typ: 'JWT' });
  const corpo = b64(payload);
  return `${cabecalho}.${corpo}.${hmac(`${cabecalho}.${corpo}`, segredo)}`;
}
```

```
function verificar(token, segredo) {
  const partes = String(token || '').split('.');
  if (partes.length !== 3) throw new Error('token malformado');
  const [cabecalho, corpo, assinatura] = partes;
  const esperado = Buffer.from(hmac(`${cabecalho}.${corpo}`, segredo));
  const recebido = Buffer.from(assinatura);
  // comparação em tempo constante (evita ataque de temporização)
  if (esperado.length !== recebido.length || !crypto.timingSafeEqual(esperado, recebido)) {
    throw new Error('assinatura invalida (token adulterado ou de outro emissor)');
  }
  const payload = JSON.parse(Buffer.from(corpo, 'base64url').toString());
  if (!payload.exp || Math.floor(Date.now() / 1000) >= payload.exp) throw new Error('token
expirado');
  return payload;
}

// Apenas lê as claims (NÃO valida) — usado pelo cliente para exibir na tela
function decodificar(token) {
  try {
    return JSON.parse(Buffer.from(String(token).split('.')[1], 'base64url').toString());
  } catch {
    return null;
  }
}

module.exports = { assinar, verificar, decodificar };
```

Resultado esperado: Arquivos salvos sem erros de sintaxe (sem sublinhado vermelho).

✔ EXEMPLO DE SUCESSO

```
node --check src\util.js; node --check src\jwt.js   (sem saída = sem erro de
sintaxe)
```

✘ EXEMPLO DE FALHA TÍPICA

```
src\jwt.js:12
  return `${cabecalho}.${corpo}.${hmac(...)
SyntaxError: Unterminated template literal
```

Como corrigir: A crase de fechamento (`) se perdeu ao copiar; copie o arquivo novamente pelo botão Copiar.

Passo 7 Criar o Servidor de Autorização

VS Code

Endpoints: `GET/POST /authorize` (login e consentimento, com validação de `redirect_uri`, escopo, `state` e PKCE), `POST /token` (authorization_code, refresh_token com rotação e client_credentials), `POST /revoke` e `/.well-known/oauth-authorization-server`.

`src/auth-server.js` (240 linhas)

src/auth-server.js

```
// ============================================================
// auth-server.js — SERVIDOR DE AUTORIZAÇÃO (porta 4000)
// Implementa: Authorization Code + PKCE, Refresh Token (com
// rotação e detecção de reuso), Client Credentials e revogação.
// ============================================================
const http = require('node:http');
const cfg = require('./config');
const jwt = require('./jwt');
const { lerCorpo, json, html, redirecionar, aleatorio, sha256b64url, iguaisSeguro, esc, pagina } =
require('./util');

const codigos = new Map(); // code -> { client_id, redirect_uri, escopos, sub, code_challenge, exp
}
const refreshTokens = new Map(); // refresh -> { sub, client_id, escopos, exp, familia }
const refreshUsados = new Map(); // refresh já trocado -> familia (para detectar reuso)

const agora = () => Math.floor(Date.now() / 1000);
const buscarCliente = (id) => cfg.CLIENTES.find((c) => c.client_id === id);
const log = (...a) => { if (!process.env.OAUTH_QUIET) console.log('[auth]', ...a); };

function validarEscopo(cliente, scope) {
  const pedidos = String(scope || '').split(' ').filter(Boolean);
  if (!pedidos.length) return null;
  return pedidos.every((e) => cliente.escopos_permitidos.includes(e)) ? pedidos : null;
}

function autenticarCliente(req, corpo) {
  let id = corpo.client_id;
  let segredo = corpo.client_secret;
  const auth = req.headers.authorization || '';
  if (auth.startsWith('Basic ')) {
    const [i, s] = Buffer.from(auth.slice(6), 'base64').toString().split(':');
    id = decodeURIComponent(i || '');
```

```
    segredo = decodeURIComponent(s || '');
  }
  const cliente = buscarCliente(id);
  return cliente && iguaisSeguro(cliente.client_secret, segredo) ? cliente : null;
}

function emitirAccessToken(sub, client_id, escopos) {
  const iat = agora();
  return jwt.assinar(
    { iss: cfg.EMISSOR, aud: cfg.AUDIENCIA_API, sub, client_id, scope: escopos.join(' '), iat, exp:
iat + cfg.ACCESS_TOKEN_TTL, jti: aleatorio(8) },
    cfg.SEGREDO_ASSINATURA
  );
}

function emitirTokens(sub, client_id, escopos, comRefresh, familia = aleatorio(8)) {
  const r = { access_token: emitirAccessToken(sub, client_id, escopos), token_type: 'Bearer',
expires_in: cfg.ACCESS_TOKEN_TTL, scope: escopos.join(' ') };
  if (comRefresh) {
    r.refresh_token = aleatorio(32); // opaco: só o provedor sabe o que significa
    r.refresh_expires_in = cfg.REFRESH_TOKEN_TTL;
    refreshTokens.set(r.refresh_token, { sub, client_id, escopos, exp: agora() +
cfg.REFRESH_TOKEN_TTL, familia });
  }
  return r;
}

function revogarFamilia(familia) {
  for (const [token, d] of refreshTokens) if (d.familia === familia) refreshTokens.delete(token);
}

// ---------- /authorize ------------------------------------------------
function validarPedido(p) {
  const cliente = buscarCliente(p.client_id);
  if (!cliente) return { fatal: 'client_id desconhecido: a aplicação não está registrada no
provedor.' };
  if (!cliente.redirect_uris.includes(p.redirect_uri)) {
    return { fatal: `redirect_uri não registrada para "${cliente.client_id}": ${p.redirect_uri}.
Por segurança o provedor NÃO redireciona para URLs desconhecidas (evita roubo do código).` };
  }
  const erro = (error, error_description) => ({ cliente, erro: { error, error_description } });
  if (!cliente.grant_types.includes('authorization_code')) return erro('unauthorized_client',
'Cliente não habilitado para o Authorization Code Flow.');
  if (p.response_type !== 'code') return erro('unsupported_response_type', 'Use response_type=code.
```

```
O Implicit Flow (response_type=token) é desaconselhado pelas boas práticas atuais.');
  const escopos = validarEscopo(cliente, p.scope);
  if (!escopos) return erro('invalid_scope', 'Escopo ausente ou não permitido para este cliente.');
  if (!p.code_challenge || p.code_challenge_method !== 'S256') return erro('invalid_request', 'PKCE
obrigatório: envie code_challenge com code_challenge_method=S256.');
  if (!p.state) return erro('invalid_request', 'Parâmetro state obrigatório (proteção contra
CSRF).');
  return { cliente, escopos };
}

function voltarComErro(res, p, erro) {
  const u = new URL(p.redirect_uri);
  u.searchParams.set('error', erro.error);
  u.searchParams.set('error_description', erro.error_description);
  if (p.state) u.searchParams.set('state', p.state);
  redirecionar(res, u.toString());
}

function paginaErro(msg) {
  return pagina('Provedor OAuth — erro', `<p class="tag">Servidor de Autorização · porta
${cfg.PORTAS.auth}</p>
  <h1>Pedido de autorização recusado</h1><p class="erro">${esc(msg)}</p>
  <a class="btn sec" href="http://localhost:${cfg.PORTAS.cliente}/">Voltar ao App Finanças</a>`,
'#7c3aed');
}

function telaConsentimento(p, cliente, escopos, mensagem = '') {
  const campos = ['client_id', 'redirect_uri', 'response_type', 'scope', 'state', 'code_challenge',
'code_challenge_method']
    .map((k) => `<input type="hidden" name="${k}" value="${esc(p[k])}">`).join('');
  return pagina('Provedor OAuth — login e consentimento', `
  <p class="tag">Servidor de Autorização · porta ${cfg.PORTAS.auth}</p>
  <h1>Entrar no Provedor OAuth</h1>
  <p>A aplicação <b>${esc(cliente.nome)}</b> (<code>${esc(cliente.client_id)}</code>) solicita
acesso a:</p>
  <ul>${escopos.map((e) => `<li><code>${esc(e)}</code> — ${esc(cfg.ESCOPOS[e] || '')}
</li>`).join('')}</ul>
  ${mensagem ? `<p class="erro">${esc(mensagem)}</p>` : ''}
  <form method="post" action="/authorize" class="bloco">${campos}
    <label>Usuário <input name="login" value="aluno" autocomplete="username"></label>
    <label>Senha <input name="senha" type="password" placeholder="senha123" autocomplete="current-
password"></label>
    <p class="dica">A senha é digitada <b>somente aqui, no provedor</b>. O App Finanças nunca a
recebe.</p>
```

```
    <button name="decisao" value="permitir">Permitir acesso</button>
    <button name="decisao" value="negar" class="sec">Negar</button>
  </form>`, '#7c3aed');
}

function getAuthorize(res, url) {
  const p = Object.fromEntries(url.searchParams);
  const v = validarPedido(p);
  if (v.fatal) return html(res, 400, paginaErro(v.fatal));
  if (v.erro) return voltarComErro(res, p, v.erro);
  html(res, 200, telaConsentimento(p, v.cliente, v.escopos));
}

async function postAuthorize(req, res) {
  const p = await lerCorpo(req);
  const v = validarPedido(p);
  if (v.fatal) return html(res, 400, paginaErro(v.fatal));
  if (v.erro) return voltarComErro(res, p, v.erro);
  if (p.decisao === 'negar') return voltarComErro(res, p, { error: 'access_denied',
error_description: 'O usuário negou o acesso.' });
  const usuario = cfg.USUARIOS.find((u) => u.login === p.login && iguaisSeguro(u.senha, p.senha));
  if (!usuario) return html(res, 401, telaConsentimento(p, v.cliente, v.escopos, 'Usuário ou senha
inválidos.'));
  const code = aleatorio(24);
  codigos.set(code, { client_id: p.client_id, redirect_uri: p.redirect_uri, escopos: v.escopos,
sub: usuario.id, code_challenge: p.code_challenge, exp: agora() + cfg.CODE_TTL });
  log(`código emitido para ${p.client_id} (usuário ${usuario.login}, escopos: ${v.escopos.join('
')})`);
  const u = new URL(p.redirect_uri);
  u.searchParams.set('code', code);
  u.searchParams.set('state', p.state);
  redirecionar(res, u.toString());
}

// ---------- /token ----------------------------------------------------
async function postToken(req, res) {
  const corpo = await lerCorpo(req);
  const erroGrant = (d) => json(res, 400, { error: 'invalid_grant', error_description: d });
  const suportados = ['authorization_code', 'refresh_token', 'client_credentials'];
  if (!suportados.includes(corpo.grant_type)) return json(res, 400, { error:
'unsupported_grant_type', error_description: `Use: ${suportados.join(', ')}` });

  const cliente = autenticarCliente(req, corpo);
  if (!cliente) return json(res, 401, { error: 'invalid_client', error_description: 'Falha na
```

```
autenticação do cliente (client_id/client_secret).' }, { 'WWW-Authenticate': 'Basic realm="oauth"' });
  if (!cliente.grant_types.includes(corpo.grant_type)) return json(res, 400, { error: 'unauthorized_client', error_description: `Cliente não habilitado para ${corpo.grant_type}.` });

  if (corpo.grant_type === 'authorization_code') {
    const d = codigos.get(corpo.code);
    codigos.delete(corpo.code); // uso único — descartado mesmo se a troca falhar
    if (!d || d.exp < agora()) return erroGrant('Código inválido, expirado ou já utilizado.');
    if (d.client_id !== cliente.client_id) return erroGrant('Código emitido para outro cliente.');
    if (d.redirect_uri !== corpo.redirect_uri) return erroGrant('redirect_uri diferente da usada na autorização.');
    if (!corpo.code_verifier || sha256b64url(corpo.code_verifier) !== d.code_challenge) return erroGrant('PKCE: code_verifier não confere com o code_challenge.');
    log(`tokens emitidos (authorization_code) para ${cliente.client_id}`);
    return json(res, 200, emitirTokens(d.sub, cliente.client_id, d.escopos, true));
  }

  if (corpo.grant_type === 'refresh_token') {
    const rt = corpo.refresh_token;
    if (refreshUsados.has(rt)) {
      revogarFamilia(refreshUsados.get(rt));
      log('ALERTA: reuso de refresh token — família revogada');
      return erroGrant('REUSO de refresh token detectado: todos os tokens desta sessão foram revogados. Faça login novamente.');
    }
    const d = refreshTokens.get(rt);
    if (!d || d.client_id !== cliente.client_id) return erroGrant('Refresh token inválido ou revogado.');
    if (d.exp < agora()) {
      refreshTokens.delete(rt);
      return erroGrant('Refresh token expirado: faça login novamente.');
    }
    refreshTokens.delete(rt); // ROTAÇÃO: o antigo deixa de valer...
    refreshUsados.set(rt, d.familia); // ...e passa a ser vigiado
    log(`tokens renovados (refresh_token) para ${cliente.client_id}`);
    return json(res, 200, emitirTokens(d.sub, cliente.client_id, d.escopos, true, d.familia));
  }

  // client_credentials: a aplicação age em nome PRÓPRIO (sem usuário, sem refresh)
  const escopos = validarEscopo(cliente, corpo.scope || cliente.escopos_permitidos.join(' '));
  if (!escopos) return json(res, 400, { error: 'invalid_scope', error_description: 'Escopo não permitido para este cliente.' });
  log(`token emitido (client_credentials) para ${cliente.client_id}`);
```

```
  return json(res, 200, emitirTokens(cliente.client_id, cliente.client_id, escopos, false));
}

// ---------- /revoke (RFC 7009) ----------------------------------------
async function postRevoke(req, res) {
  const corpo = await lerCorpo(req);
  if (!autenticarCliente(req, corpo)) return json(res, 401, { error: 'invalid_client' });
  const d = refreshTokens.get(corpo.token);
  if (d) revogarFamilia(d.familia);
  json(res, 200, { revogado: true }); // responde 200 mesmo se o token não existir
}

function metadados() {
  const e = cfg.EMISSOR;
  return {
    issuer: e,
    authorization_endpoint: `${e}/authorize`,
    token_endpoint: `${e}/token`,
    revocation_endpoint: `${e}/revoke`,
    response_types_supported: ['code'],
    grant_types_supported: ['authorization_code', 'refresh_token', 'client_credentials'],
    code_challenge_methods_supported: ['S256'],
    token_endpoint_auth_methods_supported: ['client_secret_basic', 'client_secret_post'],
    scopes_supported: Object.keys(cfg.ESCOPOS),
  };
}

function criarServidor() {
  return http.createServer(async (req, res) => {
    const url = new URL(req.url, cfg.EMISSOR);
    log(req.method, url.pathname);
    try {
      if (req.method === 'GET' && url.pathname === '/authorize') return getAuthorize(res, url);
      if (req.method === 'POST' && url.pathname === '/authorize') return await postAuthorize(req,
res);
      if (req.method === 'POST' && url.pathname === '/token') return await postToken(req, res);
      if (req.method === 'POST' && url.pathname === '/revoke') return await postRevoke(req, res);
      if (req.method === 'GET' && url.pathname === '/.well-known/oauth-authorization-server')
return json(res, 200, metadados());
      if (req.method === 'GET' && url.pathname === '/') {
        const lista = cfg.CLIENTES.map((c) => ({ client_id: c.client_id, nome: c.nome,
redirect_uris: c.redirect_uris, grant_types: c.grant_types, escopos_permitidos:
c.escopos_permitidos }));
        return html(res, 200, pagina('Provedor OAuth', `<p class="tag">Servidor de Autorização ·
```

```
porta ${cfg.PORTAS.auth}</p>
          <h1>Provedor OAuth (didático)</h1><h2>Aplicações registradas (segredos ocultos)</h2>
<pre>${esc(JSON.stringify(lista, null, 2))}</pre>
          <h2>Metadados</h2><pre>${esc(JSON.stringify(metadados(), null, 2))}</pre>`, '#7c3aed'));
      }
      json(res, 404, { error: 'not_found' });
    } catch (e) {
      console.error('[auth] erro:', e);
      json(res, 500, { error: 'server_error', error_description: e.message });
    }
  });
}

function iniciar(porta = cfg.PORTAS.auth) {
  return new Promise((resolve, reject) => {
    const s = criarServidor();
    s.once('error', reject);
    s.listen(porta, () => { log(`Servidor de Autorização em http://localhost:${porta}`);
resolve(s); });
  });
}

if (require.main === module) iniciar().catch((e) => { console.error(e.message); process.exit(1);
});
module.exports = { iniciar };
```

Resultado esperado: Você localiza no código onde o código de autorização vira uso único.

✔ EXEMPLO DE SUCESSO

```
node --check src\auth-server.js   (sem saída)
```

✘ EXEMPLO DE FALHA TÍPICA

```
SyntaxError: Unexpected end of input
```

Como corrigir: O arquivo foi colado incompleto: confira a última linha (module.exports = { iniciar };).

Passo 8 Criar a API Financeira (servidor de recursos)

VS Code

A função `autorizar()` valida assinatura, expiração, emissor (`iss`), audiência (`aud`) e o escopo exigido por rota. Responde **401** para token ausente/inválido e **403** para escopo

insuficiente.

`src/api-financeira.js` (99 linhas)

src/api-financeira.js

```
// ============================================================
// api-financeira.js — SERVIDOR DE RECURSOS (porta 5000)
// Valida o access token (assinatura, expiração, emissor,
// audiência) e exige o ESCOPO certo para cada rota.
// ============================================================
const http = require('node:http');
const cfg = require('./config');
const jwt = require('./jwt');
const { lerCorpo, json } = require('./util');

const log = (...a) => { if (!process.env.OAUTH_QUIET) console.log('[api]', ...a); };

const contas = {
  u1: {
    saldo: 2350.75,
    transacoes: [
      { id: 1, data: '2026-10-01', descricao: 'Salário', valor: 3200.0 },
      { id: 2, data: '2026-10-05', descricao: 'Aluguel', valor: -1100.0 },
      { id: 3, data: '2026-10-12', descricao: 'Supermercado', valor: -412.35 },
      { id: 4, data: '2026-10-20', descricao: 'Conta de energia', valor: -186.9 },
      { id: 5, data: '2026-10-28', descricao: 'Pix recebido', valor: 850.0 },
    ],
  },
};

// Middleware de autorização: devolve as claims ou responde 401/403
function autorizar(req, res, escopoExigido) {
  const auth = req.headers.authorization || '';
  if (!auth.startsWith('Bearer ')) {
    json(res, 401, { error: 'invalid_token', error_description: 'Envie o cabeçalho Authorization:
Bearer <access_token>.' }, { 'WWW-Authenticate': 'Bearer realm="api-financeira"' });
    return null;
  }
  let claims;
  try {
    claims = jwt.verificar(auth.slice(7), cfg.SEGREDO_ASSINATURA);
  } catch (e) {
    json(res, 401, { error: 'invalid_token', error_description: e.message }, { 'WWW-Authenticate':
'Bearer realm="api-financeira", error="invalid_token"' });
```

```
    return null;
  }
  if (claims.iss !== cfg.EMISSOR || claims.aud !== cfg.AUDIENCIA_API) {
    json(res, 401, { error: 'invalid_token', error_description: 'Emissor (iss) ou audiência (aud)
inválidos.' }, { 'WWW-Authenticate': 'Bearer error="invalid_token"' });
    return null;
  }
  const escopos = String(claims.scope || '').split(' ');
  if (!escopos.includes(escopoExigido)) {
    json(res, 403, { error: 'insufficient_scope', error_description: `Este recurso exige o escopo
"${escopoExigido}".`, escopos_do_token: escopos },
      { 'WWW-Authenticate': `Bearer error="insufficient_scope", scope="${escopoExigido}"` });
    return null;
  }
  return claims;
}

function criarServidor() {
  return http.createServer(async (req, res) => {
    const url = new URL(req.url, `http://localhost:${cfg.PORTAS.api}`);
    log(req.method, url.pathname);
    try {
      if (req.method === 'GET' && url.pathname === '/') {
        return json(res, 200, { api: 'API Financeira (didática)', rotas: { 'GET /saldo':
'saldo:read', 'GET /transacoes': 'transacoes:read', 'POST /transferencias': 'transacoes:write',
'GET /relatorios/resumo': 'relatorios:read' } });
      }
      if (req.method === 'GET' && url.pathname === '/saldo') {
        const c = autorizar(req, res, 'saldo:read'); if (!c) return;
        return json(res, 200, { usuario: c.sub, saldo: contas[c.sub]?.saldo ?? 0, moeda: 'BRL' });
      }
      if (req.method === 'GET' && url.pathname === '/transacoes') {
        const c = autorizar(req, res, 'transacoes:read'); if (!c) return;
        return json(res, 200, { usuario: c.sub, transacoes: contas[c.sub]?.transacoes ?? [] });
      }
      if (req.method === 'POST' && url.pathname === '/transferencias') {
        const c = autorizar(req, res, 'transacoes:write'); if (!c) return;
        const corpo = await lerCorpo(req);
        return json(res, 201, { status: 'transferência registrada', usuario: c.sub, ...corpo });
      }
      if (req.method === 'GET' && url.pathname === '/relatorios/resumo') {
        const c = autorizar(req, res, 'relatorios:read'); if (!c) return;
        const todas = Object.values(contas).flatMap((x) => x.transacoes);
        const entradas = todas.filter((t) => t.valor > 0).reduce((s, t) => s + t.valor, 0);
```

```
        const saidas = todas.filter((t) => t.valor < 0).reduce((s, t) => s + t.valor, 0);
        return json(res, 200, { solicitante: c.sub, periodo: '2026-10', entradas, saidas:
Math.abs(saidas), quantidade: todas.length });
      }
      json(res, 404, { error: 'not_found' });
    } catch (e) {
      console.error('[api] erro:', e);
      json(res, 500, { error: 'server_error' });
    }
  });
}

function iniciar(porta = cfg.PORTAS.api) {
  return new Promise((resolve, reject) => {
    const s = criarServidor();
    s.once('error', reject);
    s.listen(porta, () => { log(`API Financeira em http://localhost:${porta}`); resolve(s); });
  });
}

if (require.main === module) iniciar().catch((e) => { console.error(e.message); process.exit(1);
});
module.exports = { iniciar };
```

Resultado esperado: Você sabe qual escopo cada rota exige.

✔ EXEMPLO DE SUCESSO

```
node --check src\api-financeira.js   (sem saída)
```

✘ EXEMPLO DE FALHA TÍPICA

```
ReferenceError: jwt is not defined
```

Como corrigir: Falta a linha const jwt = require('./jwt'); no topo do arquivo.

Passo 9 Criar o cliente App Finanças e os scripts de início

VS Code

`cliente-web.js` é um cliente **confidencial**: guarda segredo, `code_verifier` e tokens no servidor; o navegador recebe só um cookie HttpOnly. Pede apenas `saldo:read transacoes:read` (menor privilégio). `iniciar.js` sobe os três servidores; `client-credentials.js` demonstra o fluxo servidor-servidor.

`src/cliente-web.js` (177 linhas)

src/cliente-web.js

```
// ============================================================
// cliente-web.js — APLICAÇÃO CLIENTE "App Finanças" (porta 3000)
// Cliente confidencial: o back-end guarda client_secret, tokens e
// code_verifier; o navegador recebe apenas um cookie de sessão.
// ============================================================
const http = require('node:http');
const cfg = require('./config');
const jwt = require('./jwt');
const { lerCookies, html, redirecionar, aleatorio, sha256b64url, esc, pagina } = require('./util');

const AUTH = cfg.EMISSOR;
const API = `http://localhost:${cfg.PORTAS.api}`;
const CLIENTE = cfg.CLIENTES.find((c) => c.client_id === 'app-financas-web');
const REDIRECT_URI = CLIENTE.redirect_uris[0];
const ESCOPOS_PEDIDOS = 'saldo:read transacoes:read'; // menor privilégio: NÃO pede
transacoes:write
const sessoes = new Map();
const log = (...a) => { if (!process.env.OAUTH_QUIET) console.log('[cliente]', ...a); };

function obterSessao(req, res) {
  let sid = lerCookies(req).sid;
  if (!sid || !sessoes.has(sid)) {
    sid = aleatorio(16);
    sessoes.set(sid, { log: [] });
    res.setHeader('Set-Cookie', `sid=${sid}; HttpOnly; SameSite=Lax; Path=/`);
  }
  return sessoes.get(sid);
}

async function pedirToken(params) {
  const basic =
Buffer.from(`${encodeURIComponent(CLIENTE.client_id)}:${encodeURIComponent(CLIENTE.client_secret)}`
).toString('base64');
  const r = await fetch(`${AUTH}/token`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/x-www-form-urlencoded', Authorization: `Basic ${basic}`
},
    body: new URLSearchParams(params),
  });
  return { status: r.status, corpo: await r.json() };
}
```

```
const resumir = (t) => (t ? `${t.slice(0, 16)}…` : undefined);
function guardarTokens(s, corpo) {
  s.refreshAnterior = s.tokens?.refresh_token;
  s.tokens = corpo;
}

function telaInicial(s) {
  const t = s.tokens;
  const claims = t ? jwt.decodificar(t.access_token) : null;
  const restante = claims ? claims.exp - Math.floor(Date.now() / 1000) : 0;
  const blocoLogin = `<p>Você ainda não autorizou o App Finanças a acessar sua conta.</p>
    <a class="btn" href="/login">Entrar com o Provedor OAuth</a>
    <a class="btn perigo" href="/login-redirect-invalido">Testar redirect_uri NÃO registrada</a>`;
  const blocoLogado = () => `<h2>Access token — claims (decodificado, sem validar)</h2>
    <pre>${esc(JSON.stringify(claims, null, 2))}</pre>
    <p>Expira em: <b>${restante > 0 ? `${restante} s` : 'EXPIRADO'}</b> · refresh token:
<code>${esc(resumir(t.refresh_token))}</code></p>
    <div>
      <a class="btn" href="/chamar/saldo">GET /saldo</a>
      <a class="btn" href="/chamar/transacoes">GET /transacoes</a>
      <form method="post" action="/transferir"><button class="perigo">POST /transferencias (escopo
não concedido)</button></form>
    </div><div>
      <form method="post" action="/renovar"><button class="sec">Renovar token (refresh)</button>
</form>
      ${s.refreshAnterior ? '<form method="post" action="/reusar-refresh"><button
class="perigo">Reutilizar refresh token ANTIGO</button></form>' : ''}
      <form method="post" action="/sair"><button class="sec">Sair (revogar)</button></form>
      <a class="btn sec" href="/">Atualizar</a>
    </div>`;
  const u = s.ultimo;
  return pagina('App Finanças — cliente OAuth', `
    <p class="tag">Aplicação cliente · App Finanças · porta ${cfg.PORTAS.cliente}</p>
    <h1>App Finanças</h1>
    ${t ? blocoLogado() : blocoLogin}
    ${u ? `<h2>Última operação: ${esc(u.titulo)} → HTTP ${u.status}</h2>
<pre>${esc(JSON.stringify(u.corpo, null, 2))}</pre>` : ''}
    ${s.log.length ? `<h2>Linha do tempo do fluxo</h2><ol>${s.log.map((l) => `<li>${esc(l)}
</li>`).join('')}</ol>` : ''}`);
}

function urlAutorizacao(s, redirectUri) {
  s.state = aleatorio(16); // anti-CSRF
  s.verifier = aleatorio(32); // PKCE: segredo que só o back-end do cliente conhece
```

```
  const challenge = sha256b64url(s.verifier);
  const u = new URL(`${AUTH}/authorize`);
  u.search = new URLSearchParams({ response_type: 'code', client_id: CLIENTE.client_id,
redirect_uri: redirectUri, scope: ESCOPOS_PEDIDOS, state: s.state, code_challenge: challenge,
code_challenge_method: 'S256' });
  s.log = [
    `Cliente gerou state=${s.state.slice(0, 8)}… e um code_verifier secreto; calculou
code_challenge=${challenge.slice(0, 10)}… (SHA-256).`,
    `Navegador redirecionado para ${AUTH}/authorize pedindo os escopos "${ESCOPOS_PEDIDOS}".`,
  ];
  return u.toString();
}

async function callback(s, url, res) {
  const q = url.searchParams;
  if (q.get('error')) {
    s.ultimo = { titulo: 'Autorização não concedida', status: 400, corpo: Object.fromEntries(q) };
    s.log.push(`Provedor devolveu erro: ${q.get('error')}.`);
    return redirecionar(res, '/');
  }
  if (!s.state || q.get('state') !== s.state) {
    return html(res, 400, pagina('Erro', '<h1>state inválido</h1><p class="erro">Possível ataque
CSRF: esta resposta não corresponde a um pedido iniciado nesta sessão.</p><a class="btn"
href="/">Voltar</a>'));
  }
  const code = q.get('code');
  s.log.push(`Callback recebeu code=${code.slice(0, 8)}… e o state conferiu.`);
  const r = await pedirToken({ grant_type: 'authorization_code', code, redirect_uri: REDIRECT_URI,
code_verifier: s.verifier });
  delete s.state;
  delete s.verifier;
  if (r.status !== 200) {
    s.ultimo = { titulo: 'Troca do código falhou', ...r };
    return redirecionar(res, '/');
  }
  s.log.push('Back-end do cliente trocou code + code_verifier + client_secret por tokens em POST
/token (canal servidor-servidor).');
  guardarTokens(s, r.corpo);
  s.ultimo = { titulo: 'Tokens recebidos (valores abreviados)', status: 200, corpo: { ...r.corpo,
access_token: resumir(r.corpo.access_token), refresh_token: resumir(r.corpo.refresh_token) } };
  redirecionar(res, '/');
}

async function chamarApi(s, metodo, caminho, corpo) {
```

```
  const r = await fetch(`${API}${caminho}`, {
    method: metodo,
    headers: { Authorization: `Bearer ${s.tokens.access_token}`, 'Content-Type': 'application/json'
},
    body: corpo ? JSON.stringify(corpo) : undefined,
  });
  s.ultimo = { titulo: `${metodo} ${caminho}`, status: r.status, corpo: await r.json() };
  s.log.push(`Chamou ${metodo} ${caminho} com Authorization: Bearer → HTTP ${r.status}.`);
}

async function renovar(s, refresh, titulo) {
  const r = await pedirToken({ grant_type: 'refresh_token', refresh_token: refresh });
  if (r.status === 200) {
    guardarTokens(s, r.corpo);
    s.ultimo = { titulo, status: 200, corpo: { ...r.corpo, access_token:
resumir(r.corpo.access_token), refresh_token: resumir(r.corpo.refresh_token) } };
    s.log.push('Refresh token trocado por um NOVO par de tokens (rotação: o refresh antigo foi
invalidado).');
  } else {
    s.ultimo = { titulo, ...r };
    s.log.push(`Renovação recusada: ${r.corpo.error_description}`);
    if (String(r.corpo.error_description).includes('REUSO')) delete s.tokens; // sessão
comprometida
  }
}

function criarServidor() {
  return http.createServer(async (req, res) => {
    const url = new URL(req.url, `http://localhost:${cfg.PORTAS.cliente}`);
    const s = obterSessao(req, res);
    log(req.method, url.pathname);
    try {
      if (req.method === 'GET' && url.pathname === '/') return html(res, 200, telaInicial(s));
      if (req.method === 'GET' && url.pathname === '/login') return redirecionar(res,
urlAutorizacao(s, REDIRECT_URI));
      if (req.method === 'GET' && url.pathname === '/login-redirect-invalido') return
redirecionar(res, urlAutorizacao(s, 'http://site-malicioso.exemplo/callback'));
      if (req.method === 'GET' && url.pathname === '/callback') return await callback(s, url, res);
      if (!s.tokens) return redirecionar(res, '/'); // rotas abaixo exigem login
      if (req.method === 'GET' && url.pathname === '/chamar/saldo') await chamarApi(s, 'GET',
'/saldo');
      else if (req.method === 'GET' && url.pathname === '/chamar/transacoes') await chamarApi(s,
'GET', '/transacoes');
      else if (req.method === 'POST' && url.pathname === '/transferir') await chamarApi(s, 'POST',
```

```
'/transferencias', { destino: 'Conta 9999-0', valor: 100 });
      else if (req.method === 'POST' && url.pathname === '/renovar') await renovar(s,
s.tokens.refresh_token, 'Renovação com refresh token atual');
      else if (req.method === 'POST' && url.pathname === '/reusar-refresh' && s.refreshAnterior)
await renovar(s, s.refreshAnterior, 'Tentativa de REUSO do refresh token antigo');
      else if (req.method === 'POST' && url.pathname === '/sair') {
        const basic =
Buffer.from(`${CLIENTE.client_id}:${CLIENTE.client_secret}`).toString('base64');
        await fetch(`${AUTH}/revoke`, { method: 'POST', headers: { Authorization: `Basic ${basic}`,
'Content-Type': 'application/x-www-form-urlencoded' }, body: new URLSearchParams({ token:
s.tokens.refresh_token }) });
        delete s.tokens;
        delete s.refreshAnterior;
        s.ultimo = { titulo: 'Logout', status: 200, corpo: { mensagem: 'Refresh token revogado no
provedor e sessão local encerrada.' } };
        s.log = [];
      }
      redirecionar(res, '/'); // padrão Post/Redirect/Get
    } catch (e) {
      console.error('[cliente] erro:', e);
      html(res, 502, pagina('Erro', `<h1>Falha ao falar com o provedor/API</h1>
<pre>${esc(e.message)}</pre><p>Os três servidores estão rodando? Use <code>npm start</code>.
</p>`));
    }
  });
}

function iniciar(porta = cfg.PORTAS.cliente) {
  return new Promise((resolve, reject) => {
    const s = criarServidor();
    s.once('error', reject);
    s.listen(porta, () => { log(`App Finanças em http://localhost:${porta}`); resolve(s); });
  });
}

if (require.main === module) iniciar().catch((e) => { console.error(e.message); process.exit(1);
});
module.exports = { iniciar };
```

`src/iniciar.js` (27 linhas)

src/iniciar.js

```
// ============================================================
// iniciar.js — Sobe os três papéis do OAuth no mesmo processo
```

```
// ============================================================
const cfg = require('./config');
const auth = require('./auth-server');
const api = require('./api-financeira');
const cliente = require('./cliente-web');

(async () => {
  await auth.iniciar();
  await api.iniciar();
  await cliente.iniciar();
  console.log(`
============================================================
 Laboratório OAuth em execução
  Servidor de Autorização : ${cfg.EMISSOR}
  API Financeira (recursos): http://localhost:${cfg.PORTAS.api}
  App Finanças (cliente)   : http://localhost:${cfg.PORTAS.cliente}   <-- abra no navegador
  Usuário demo: aluno / senha123   | access token: ${cfg.ACCESS_TOKEN_TTL}s
  Pressione Ctrl+C para encerrar.
============================================================`);
})().catch((e) => {
  console.error(`Falha ao iniciar: ${e.message}`);
  if (e.code === 'EADDRINUSE') console.error('Porta ocupada. Feche o outro servidor (Ctrl+C) ou
defina AUTH_PORT/API_PORT/CLIENT_PORT.');
  process.exit(1);
});
```

`src/client-credentials.js` (39 linhas)

src/client-credentials.js

```
// ============================================================
// client-credentials.js — Comunicação SERVIDOR-SERVIDOR
// O "Serviço de Relatórios" não tem usuário: autentica-se com o
// próprio client_id/client_secret e recebe um token só dele.
// Uso (com "npm start" rodando em outro terminal): npm run cc
// ============================================================
const cfg = require('./config');

const svc = cfg.CLIENTES.find((c) => c.client_id === 'servico-relatorios');
const API = `http://localhost:${cfg.PORTAS.api}`;
const basic = Buffer.from(`${svc.client_id}:${svc.client_secret}`).toString('base64');

async function chamar(caminho, token) {
  const r = await fetch(`${API}${caminho}`, { headers: { Authorization: `Bearer ${token}` } });
  console.log(`   GET ${caminho} -> HTTP ${r.status}`);
```

```
  console.log(JSON.stringify(await r.json(), null, 2).replace(/^/gm, '    '));
}

(async () => {
  console.log('1) Serviço de Relatórios pede token direto ao provedor
(grant_type=client_credentials)');
  const r = await fetch(`${cfg.EMISSOR}/token`, {
    method: 'POST',
    headers: { Authorization: `Basic ${basic}`, 'Content-Type': 'application/x-www-form-urlencoded'
},
    body: new URLSearchParams({ grant_type: 'client_credentials', scope: 'relatorios:read' }),
  });
  const t = await r.json();
  console.log(`   HTTP ${r.status} | scope=${t.scope} | expires_in=${t.expires_in}s |
refresh_token=${t.refresh_token ?? '(nenhum — não há usuário)'}`);
  if (!r.ok) process.exit(1);

  console.log('2) Usa o token no recurso permitido pelo escopo:');
  await chamar('/relatorios/resumo', t.access_token);

  console.log('3) Tenta um recurso de usuário (fora do escopo) — esperado 403:');
  await chamar('/transacoes', t.access_token);
})().catch((e) => {
  console.error(`Erro: ${e.message}. O laboratório está rodando? Execute "npm start" em outro
terminal.`);
  process.exit(1);
});
```

Resultado esperado: Sete arquivos dentro de `src/` .

✔ EXEMPLO DE SUCESSO

```
node --check src\cliente-web.js   (sem saída)
```

✘ EXEMPLO DE FALHA TÍPICA

```
Error: Cannot find module './util'
```

Como corrigir: util.js precisa estar na mesma pasta src\ (passo 6).

Passo 10 Criar os testes automatizados

VS Code + PowerShell

Os testes sobem os servidores em portas de teste (13000/14000/15000) e verificam 25 regras de segurança. O arquivo `.http` é usado com a extensão REST Client.

VS Code + PowerShell — comandos

```
npm test
```

`testes/teste-fluxo.js` (122 linhas)

testes/teste-fluxo.js

```
// ============================================================
// teste-fluxo.js — Testes automáticos do laboratório (npm test)
// Sobe os servidores em portas de teste e verifica cada regra de
// segurança do OAuth. Saída: [OK] / [FALHA] e código de saída.
// ============================================================
process.env.AUTH_PORT = '14000';
process.env.API_PORT = '15000';
process.env.CLIENT_PORT = '13000';
process.env.OAUTH_QUIET = '1';

const cfg = require('../src/config');
const jwt = require('../src/jwt');
const { sha256b64url, aleatorio } = require('../src/util');

const AUTH = cfg.EMISSOR;
const API = `http://localhost:${cfg.PORTAS.api}`;
const WEB = cfg.CLIENTES[0];
const SVC = cfg.CLIENTES[1];
const REDIRECT = WEB.redirect_uris[0];
const basic = (c) => `Basic
${Buffer.from(`${c.client_id}:${c.client_secret}`).toString('base64')}`;

let ok = 0;
let falhas = 0;
function verificar(descricao, condicao, detalhe = '') {
  if (condicao) { ok++; console.log(`  [OK]    ${descricao}`); }
  else { falhas++; console.log(`  [FALHA] ${descricao}${detalhe ? ` -> ${detalhe}` : ''}`); }
}

async function token(params, cliente = WEB) {
  const r = await fetch(`${AUTH}/token`, { method: 'POST', headers: { Authorization: basic(cliente)
}, body: new URLSearchParams(params) });
  return { status: r.status, corpo: await r.json() };
}

async function obterCodigo(verifier, extra = {}) {
  const corpo = new URLSearchParams({
```

```
    response_type: 'code', client_id: WEB.client_id, redirect_uri: REDIRECT, scope: 'saldo:read
transacoes:read',
    state: 'estado123', code_challenge: sha256b64url(verifier), code_challenge_method: 'S256',
    login: 'aluno', senha: 'senha123', decisao: 'permitir', ...extra,
  });
  const r = await fetch(`${AUTH}/authorize`, { method: 'POST', body: corpo, redirect: 'manual' });
  const loc = r.headers.get('location');
  return loc ? new URL(loc).searchParams : null;
}

const getApi = (caminho, tk) => fetch(`${API}${caminho}`, { headers: tk ? { Authorization: `Bearer
${tk}` } : {} });

(async () => {
  await require('../src/auth-server').iniciar();
  await require('../src/api-financeira').iniciar();
  await require('../src/cliente-web').iniciar();
  console.log('\nTestes do laboratório OAuth\n');

  console.log('1. Registro e descoberta');
  const meta = await (await fetch(`${AUTH}/.well-known/oauth-authorization-server`)).json();
  verificar('metadados publicam token_endpoint e PKCE S256', meta.token_endpoint &&
meta.code_challenge_methods_supported.includes('S256'));
  verificar('App Finanças responde na porta do cliente', (await
fetch(`http://localhost:${cfg.PORTAS.cliente}/`)).status === 200);

  console.log('2. Pedido de autorização');
  const ruim = await fetch(`${AUTH}/authorize?${new URLSearchParams({ response_type: 'code',
client_id: WEB.client_id, redirect_uri: 'http://site-malicioso.exemplo/cb', scope: 'saldo:read',
state: 'x', code_challenge: 'abc', code_challenge_method: 'S256' })}`, { redirect: 'manual' });
  verificar('redirect_uri não registrada é recusada SEM redirecionar (400)', ruim.status === 400 &&
!ruim.headers.get('location'));
  const semPkce = await fetch(`${AUTH}/authorize?${new URLSearchParams({ response_type: 'code',
client_id: WEB.client_id, redirect_uri: REDIRECT, scope: 'saldo:read', state: 'x' })}`, { redirect:
'manual' });
  verificar('pedido sem PKCE volta com error=invalid_request', new
URL(semPkce.headers.get('location') || 'http://x').searchParams.get('error') ===
'invalid_request');
  const implicito = await fetch(`${AUTH}/authorize?${new URLSearchParams({ response_type: 'token',
client_id: WEB.client_id, redirect_uri: REDIRECT, scope: 'saldo:read', state: 'x' })}`, { redirect:
'manual' });
  verificar('Implicit Flow (response_type=token) é recusado', new
URL(implicito.headers.get('location') || 'http://x').searchParams.get('error') ===
'unsupported_response_type');
```

```
  const escopoIndevido = await obterCodigo('v'.repeat(43), { scope: 'relatorios:read' });
  verificar('escopo não permitido ao cliente -> invalid_scope', escopoIndevido?.get('error') ===
'invalid_scope');
  const negado = await obterCodigo('v'.repeat(43), { decisao: 'negar' });
  verificar('usuário que nega o consentimento -> access_denied', negado?.get('error') ===
'access_denied');

  console.log('3. Troca do código (Authorization Code + PKCE)');
  const v1 = aleatorio(32);
  const q1 = await obterCodigo(v1);
  verificar('login correto devolve code e o mesmo state', q1?.get('code') && q1.get('state') ===
'estado123');
  const pkceErrado = await token({ grant_type: 'authorization_code', code: q1.get('code'),
redirect_uri: REDIRECT, code_verifier: 'verificador-errado' });
  verificar('code_verifier errado -> invalid_grant', pkceErrado.status === 400 &&
pkceErrado.corpo.error === 'invalid_grant');
  const v2 = aleatorio(32);
  const q2 = await obterCodigo(v2);
  const segredoErrado = await token({ grant_type: 'authorization_code', code: q2.get('code'),
redirect_uri: REDIRECT, code_verifier: v2 }, { ...WEB, client_secret: 'errado' });
  verificar('client_secret errado -> 401 invalid_client', segredoErrado.status === 401);
  const v3 = aleatorio(32);
  const q3 = await obterCodigo(v3);
  const t1 = await token({ grant_type: 'authorization_code', code: q3.get('code'), redirect_uri:
REDIRECT, code_verifier: v3 });
  verificar('troca correta devolve access_token + refresh_token', t1.status === 200 &&
t1.corpo.access_token && t1.corpo.refresh_token);
  verificar('token traz apenas os escopos pedidos (menor privilégio)', t1.corpo.scope ===
'saldo:read transacoes:read');
  const reuso = await token({ grant_type: 'authorization_code', code: q3.get('code'), redirect_uri:
REDIRECT, code_verifier: v3 });
  verificar('código é de uso único (2ª troca -> invalid_grant)', reuso.status === 400);

  console.log('4. Servidor de recursos (API)');
  verificar('GET /transacoes com token válido -> 200', (await getApi('/transacoes',
t1.corpo.access_token)).status === 200);
  verificar('GET /transacoes sem token -> 401', (await getApi('/transacoes')).status === 401);
  const transf = await fetch(`${API}/transferencias`, { method: 'POST', headers: { Authorization:
`Bearer ${t1.corpo.access_token}` } });
  verificar('POST /transferencias sem escopo transacoes:write -> 403', transf.status === 403);
  const [h, , s] = t1.corpo.access_token.split('.');
  const pAdulterado = Buffer.from(JSON.stringify({ ...jwt.decodificar(t1.corpo.access_token),
scope: 'transacoes:write' })).toString('base64url');
  verificar('token adulterado (escopo alterado) -> 401', (await getApi('/transacoes',
```

```
`${h}.${pAdulterado}.${s}`)).status === 401);
  const expirado = jwt.assinar({ ...jwt.decodificar(t1.corpo.access_token), exp:
Math.floor(Date.now() / 1000) - 5 }, cfg.SEGREDO_ASSINATURA);
  verificar('token expirado -> 401', (await getApi('/transacoes', expirado)).status === 401);

  console.log('5. Renovação (refresh token com rotação)');
  const t2 = await token({ grant_type: 'refresh_token', refresh_token: t1.corpo.refresh_token });
  verificar('refresh válido devolve NOVO par de tokens', t2.status === 200 &&
t2.corpo.refresh_token !== t1.corpo.refresh_token);
  const reusoRt = await token({ grant_type: 'refresh_token', refresh_token: t1.corpo.refresh_token
});
  verificar('reuso do refresh antigo é detectado (invalid_grant)', reusoRt.status === 400 &&
reusoRt.corpo.error_description.includes('REUSO'));
  const t3 = await token({ grant_type: 'refresh_token', refresh_token: t2.corpo.refresh_token });
  verificar('após reuso, o refresh novo também foi revogado (família)', t3.status === 400);

  console.log('6. Client Credentials (servidor-servidor)');
  const cc = await token({ grant_type: 'client_credentials', scope: 'relatorios:read' }, SVC);
  verificar('serviço obtém token sem usuário e sem refresh_token', cc.status === 200 &&
!cc.corpo.refresh_token);
  verificar('GET /relatorios/resumo com token do serviço -> 200', (await
getApi('/relatorios/resumo', cc.corpo.access_token)).status === 200);
  verificar('token do serviço não acessa /transacoes -> 403', (await getApi('/transacoes',
cc.corpo.access_token)).status === 403);
  const ccWeb = await token({ grant_type: 'client_credentials' }, WEB);
  verificar('App Finanças não pode usar client_credentials -> unauthorized_client',
ccWeb.corpo.error === 'unauthorized_client');

  console.log(`\nResultado: ${ok} OK, ${falhas} FALHA(S)\n`);
  process.exit(falhas ? 1 : 0);
})().catch((e) => {
  console.error(`Erro inesperado nos testes: ${e.stack || e.message}`);
  if (e.code === 'EADDRINUSE') console.error('Portas de teste 13000/14000/15000 ocupadas.');
  process.exit(2);
});
```

`testes/requisicoes.http` (32 linhas)

testes/requisicoes.http

```
# Requisições para a extensão REST Client (VS Code) — clique em "Send Request"
# Pré-requisito: "npm start" rodando.

### 1) Metadados do provedor (descoberta)
GET http://localhost:4000/.well-known/oauth-authorization-server
```

```
### 2) Client Credentials — token do Serviço de Relatórios
# @name tokenServico
POST http://localhost:4000/token
Authorization: Basic servico-relatorios:segredo-relatorios
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials&scope=relatorios:read

### 3) Relatório com o token do serviço (esperado 200)
GET http://localhost:5000/relatorios/resumo
Authorization: Bearer {{tokenServico.response.body.access_token}}

### 4) Recurso de usuário com token do serviço (esperado 403 insufficient_scope)
GET http://localhost:5000/transacoes
Authorization: Bearer {{tokenServico.response.body.access_token}}

### 5) Sem token (esperado 401 invalid_token)
GET http://localhost:5000/saldo

### 6) Segredo errado (esperado 401 invalid_client)
POST http://localhost:4000/token
Authorization: Basic servico-relatorios:segredo-errado
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials&scope=relatorios:read
```

Resultado esperado: Saída termina em `Resultado: 25 OK, 0 FALHA(S)`.

✔ EXEMPLO DE SUCESSO

```
  [OK]    código é de uso único (2ª troca -> invalid_grant)
...
Resultado: 25 OK, 0 FALHA(S)
```

✘ EXEMPLO DE FALHA TÍPICA

```
  [FALHA] POST /transferencias sem escopo transacoes:write -> 403
Resultado: 24 OK, 1 FALHA(S)
```

Como corrigir: Leia a linha [FALHA]: ela aponta a regra quebrada; compare o arquivo correspondente com o código da página.

Passo 11 Copiar o modelo do plano e fazer o primeiro commit

VS Code + Git

Crie `docs/plano-oauth.md` com o modelo abaixo (será completado no passo 17) e registre a versão inicial.

VS Code + Git — comandos

```
git add .
git commit -m "feat: laboratorio OAuth (auth code + PKCE, refresh e client credentials)"
git log --oneline
```

`docs/plano-oauth.md` (57 linhas)

docs/plano-oauth.md

```
# Plano de implementação de OAuth 2.0 — API Financeira

**Grupo:** _(nomes dos integrantes)_  **Turma:** _____  **Data:** ___/___/_____

## 1. Contexto
A API financeira expõe saldo, extrato e transferências. Ela é consumida por (a) um aplicativo web
usado por pessoas e (b) um serviço interno de relatórios que roda sem usuário. Objetivo: permitir
esse acesso **sem que nenhuma aplicação conheça a senha do usuário**, com permissões mínimas e
tokens de vida curta.

## 2. Escolha do fluxo (justificativa)
| Cliente | Fluxo escolhido | Por quê |
|---|---|---|
| App Finanças (web, com back-end) | **Authorization Code + PKCE** | O usuário se autentica só no
provedor; o código trafega pelo navegador mas só vale junto com o `code_verifier` (PKCE) e o
`client_secret`, que ficam no back-end. É o fluxo recomendado pelas boas práticas atuais (RFC 9700
/ OAuth 2.1). |
| SPA ou app mobile (futuro) | **Authorization Code + PKCE** (cliente público, sem segredo) | O
**Implicit Flow** foi descontinuado: entrega o token na URL, expondo-o a histórico, logs e scripts.
|
| Serviço de Relatórios | **Client Credentials** | Comunicação servidor-servidor: não existe
usuário, a aplicação age em nome próprio. Não recebe refresh token. |

## 3. Configuração
1. **Registro da aplicação** no provedor (no laboratório: `src/config.js`): `client_id`,
`client_secret` (guardado em variável de ambiente/cofre, nunca no Git), tipo de cliente e fluxos
permitidos (`grant_types`).
2. **URLs de redirecionamento**: lista exata, sempre HTTPS em produção
```

(`https://app.exemplo.com.br/callback`), sem curingas. O provedor recusa qualquer outra — no lab, `http://site-malicioso.exemplo/callback` é bloqueada.
3. **Escopos** (menor privilégio):

Escopo	Permite	Quem recebe
`saldo:read`	Consultar saldo	App Finanças
`transacoes:read`	Consultar extrato	App Finanças
`transacoes:write`	Transferências	Somente com consentimento explícito e, idealmente, autenticação forte (MFA)
`relatorios:read`	Relatório consolidado	Serviço de Relatórios

4. **Proteções do pedido**: `state` aleatório por pedido (anti-CSRF), PKCE `S256` obrigatório, código de autorização de **uso único** com validade de 60 s.
5. **Validação na API**: assinatura do JWT, `exp`, `iss`, `aud` e escopo por rota — 401 para token inválido/expirado, 403 para escopo insuficiente.

4. Gestão de tokens

Token	Formato	Validade	Onde fica
Access token	JWT assinado	5–15 min (lab: 2 min)	Memória do back-end do cliente
Refresh token	Opaco (aleatório)	30 min a alguns dias, conforme risco	Back-end do cliente (nunca em localStorage)
Código de autorização	Opaco	60 s, uso único	Trafega uma vez pelo navegador

Exemplo prático: às 10h00 o usuário autoriza o app e recebe access (expira 10h05) + refresh. Às 10h06 a API responde 401 → o back-end usa o refresh e recebe um **novo par** (rotação); o refresh antigo é invalidado. Se alguém reutilizar o refresh antigo, o provedor detecta o **reuso** e revoga toda a família de tokens, forçando novo login. No logout, o app chama `/revoke`.

5. Comunicação com a equipe

- Resumo de uma página + diagrama de sequência do fluxo; decisão registrada (ADR) no repositório.
- Checklist obrigatório em code review (PKCE, `state`, redirect exato, escopos mínimos, segredos fora do código, validação `iss/aud/exp`).
- Testes automatizados de segurança no CI (`npm test`) — o pipeline falha se uma regra for quebrada.
- Demonstração ao vivo no laboratório e canal único de dúvidas; revisão do plano a cada mudança de escopo.

6. Evidências do laboratório

- [ ] Print do `npm test` com 25 OK
- [ ] Print do consentimento no provedor e das claims do token
- [ ] Print do 403 (escopo) e do erro de redirect_uri
- [ ] Print da renovação e da detecção de reuso

```
- [ ] Saída do `npm run cc`


## Referências
- SEDUC-SP. *Descomplicando o OAuth: autorização segura para APIs* — Programação Back-End, 2ª
série, 4º bimestre, semana 25, aula 1.
- [RFC 6749 — The OAuth 2.0 Authorization Framework](https://www.rfc-editor.org/rfc/rfc6749)
- [RFC 7636 — PKCE](https://www.rfc-editor.org/rfc/rfc7636)
- [RFC 9700 — Best Current Practice for OAuth 2.0 Security](https://www.rfc-editor.org/rfc/rfc9700)
```

Resultado esperado: `git log` mostra o commit.

✔ EXEMPLO DE SUCESSO

```
[main (root-commit) 3f9c2ab] feat: laboratorio OAuth ...
 14 files changed, 1040 insertions(+)
```

✘ EXEMPLO DE FALHA TÍPICA

```
Author identity unknown
*** Please tell me who you are.
```

Como corrigir: git config --global user.name "Seu Nome" e git config --global user.email "seu@email"; repita o commit.

Passo 12 Iniciar os servidores

PowerShell

Suba o laboratório e deixe este terminal aberto (ele mostra os logs `[auth]`, `[api]`, `[cliente]`). Visite também a página do provedor para ver as aplicações registradas e os metadados.

PowerShell — comandos

```
npm start
# depois abra no navegador:
#   http://localhost:3000   (App Finanças)
#   http://localhost:4000   (registro + metadados do provedor)
```

Resultado esperado: Banner “Laboratório OAuth em execução” e as duas páginas abrem.

✔ EXEMPLO DE SUCESSO

```
================================================================
 Laboratório OAuth em execução
  App Finanças (cliente)   : http://localhost:3000
```

✘ EXEMPLO DE FALHA TÍPICA

```
Falha ao iniciar: listen EADDRINUSE: address already in use :::4000
Porta ocupada...
```

Como corrigir: Feche o outro terminal com npm start (Ctrl+C) ou encerre o processo: Get-NetTCPConnection -LocalPort 4000 | Select OwningProcess; Stop-Process -Id <PID>.

Passo 13 Executar o Authorization Code Flow com PKCE

Navegador

Abra o DevTools (F12) na aba **Rede** com "Preservar log". Clique em **Entrar com o Provedor OAuth** e observe na URL os parâmetros `response_type=code`, `state`, `code_challenge` e `scope`. Entre com **aluno / senha123** e clique em **Permitir acesso**. Observe o redirecionamento para `/callback?code=…&state=…`, as claims do token e a linha do tempo. Depois clique em **GET /saldo** e **GET /transacoes**.

Resultado esperado: HTTP 200 nas duas consultas; print das claims (scope = `saldo:read transacoes:read`).

✔ EXEMPLO DE SUCESSO

```
Última operação: GET /transacoes → HTTP 200
"scope": "saldo:read transacoes:read"
```

✘ EXEMPLO DE FALHA TÍPICA

```
Pedido de autorização recusado / Usuário ou senha inválidos.
```

Como corrigir: Use aluno / senha123. Se aparecer erro de redirect_uri, confira CLIENT_PORT e redirect_uris em config.js.

Passo 14 Testar as regras de segurança

Navegador + PowerShell

(a) Clique em **POST /transferencias → 403 insufficient_scope** (o app não pediu `transacoes:write`). (b) Clique em **Sair** e depois em **Testar redirect_uri NÃO registrada** → o provedor recusa sem redirecionar. (c) Faça login e clique em **Negar** → `access_denied`. (d) Em outro terminal, chame a API sem token:

Navegador + PowerShell — comandos

```
curl.exe -i http://localhost:5000/transacoes      # esperado: HTTP/1.1 401
```

Resultado esperado: Prints do 403, do erro de redirect_uri e do 401.

✔ EXEMPLO DE SUCESSO

```
POST /transferencias → HTTP 403 "insufficient_scope"
HTTP/1.1 401 Unauthorized  (curl.exe sem token)
```

✘ EXEMPLO DE FALHA TÍPICA

```
curl : Não é possível vincular o parâmetro 'Headers'...  (usou curl em vez de curl.exe)
```

Como corrigir: No Windows PowerShell 5.1, curl é apelido de Invoke-WebRequest: digite curl.exe.

Passo 15 Observar expiração, renovação e reuso de token

PowerShell + Navegador

Pare o servidor (**Ctrl+C**) e reinicie com access token de 20 s. Faça login, espere 20 s e clique em **GET /saldo → 401 token expirado**. Clique em **Renovar token** → novo par de tokens (rotação). Clique em **Reutilizar refresh token ANTIGO → 400 REUSO detectado** e a sessão é revogada.

PowerShell + Navegador — comandos

```
$env:ACCESS_TTL = 20
npm start
# ao terminar o passo: Ctrl+C e depois
Remove-Item Env:ACCESS_TTL
```

Resultado esperado: Prints do 401 por expiração, da renovação (200) e do reuso (400).

✔ EXEMPLO DE SUCESSO

```
GET /saldo → HTTP 401 "token expirado"
Renovação com refresh token atual → HTTP 200
Tentativa de REUSO ... → HTTP 400
```

✘ EXEMPLO DE FALHA TÍPICA

```
Expira em: 118 s  (a variável não foi aplicada)
```

Como corrigir: Defina $env:ACCESS_TTL = 20 no MESMO terminal antes do npm start.

Passo 16 Executar o Client Credentials Flow

PowerShell (2º terminal)

Com o servidor rodando, abra outro terminal. O Serviço de Relatórios obtém token sem usuário e sem refresh token, acessa o relatório (200) e é barrado em `/transacoes` (403).

Alternativa: abra `testes/requisicoes.http` no VS Code e use **Send Request** (REST Client).

PowerShell (2º terminal) — comandos

```
cd "$HOME\Documents\atividade-oauth-api"
npm run cc
```

Resultado esperado: Saída mostra HTTP 200 no relatório e HTTP 403 em /transacoes.

✔ EXEMPLO DE SUCESSO

```
1) ... HTTP 200 | scope=relatorios:read | refresh_token=(nenhum — não há usuário)
   GET /relatorios/resumo -> HTTP 200
   GET /transacoes -> HTTP 403
```

✘ EXEMPLO DE FALHA TÍPICA

```
Erro: fetch failed. O laboratório está rodando? Execute "npm start" em outro terminal.
```

Como corrigir: Deixe o npm start rodando no 1º terminal e execute npm run cc no 2º.

Passo 17 Redigir o plano de implementação do grupo

VS Code

Etapa **Execução** do roteiro: complete `docs/plano-oauth.md` com os nomes do grupo, adapte as justificativas com as palavras do grupo e anexe as evidências (prints) dos passos 10 e 13 a 16. O plano precisa cobrir: escolha do fluxo, configuração (registro, escopos, redirect URIs) e gestão de tokens (expiração, renovação).

Resultado esperado: As seis seções do plano preenchidas.

✔ EXEMPLO DE SUCESSO

```
docs/plano-oauth.md com as 6 seções preenchidas e prints anexados
```

✘ EXEMPLO DE FALHA TÍPICA

```
Seções 1–5 copiadas sem adaptação e sem evidências (seção 6 vazia)
```

Como corrigir: Reescreva as justificativas com as palavras do grupo e anexe os prints dos passos 10 e 13–16.

Passo 18 Responder às perguntas e entregar

Git + VS Code

Responda às quatro perguntas da mensagem do Slack (seção Respostas desta página serve de gabarito para conferência após 50% de progresso), registre o plano no Git e

entregue conforme a orientação do(a) professor(a). Se a turma usa repositório remoto, a publicação é manual: crie o repositório no GitHub, faça login e rode `git remote add origin <URL>` e `git push -u origin main`.

Git + VS Code — comandos

```
git add docs
git commit -m "docs: plano de implementacao OAuth do grupo"
git log --oneline
```

Resultado esperado: Dois commits no histórico e respostas entregues.

✔ EXEMPLO DE SUCESSO

```
8b1d0e4 docs: plano de implementacao OAuth do grupo
3f9c2ab feat: laboratorio OAuth ...
```

✘ EXEMPLO DE FALHA TÍPICA

```
nothing to commit, working tree clean
```

Como corrigir: Salve o arquivo no VS Code (Ctrl+S) antes do git add.

Rubrica de avaliação (10 pontos)

Item	Critério de certo	Pontuação	Máx.
E1 — Ambiente e testes (passos 1–10)	Print do `npm test` com 25 OK, 0 FALHA(S).	Certo: 1,0 · Errado: 0	**1,0**
E2 — Authorization Code + PKCE (passo 13)	Print das claims com `scope` = `saldo:read transacoes:read` e da linha do tempo do fluxo.	Certo: 1,0 · Errado: 0	**1,0**
E3 — Regras de segurança (passo 14)	Prints do 403 `insufficient_scope` e da recusa da redirect_uri.	0,5 por evidência	**1,0**
E4 — Renovação e Client Credentials (15–16)	Print do reuso detectado (400) e saída do `npm run cc` com 200 e 403.	0,5 por evidência	**1,0**
E5 — Plano de implementação (17–18)	`docs/plano-oauth.md` com nomes, fluxo justificado, configuração (registro + escopos + redirect),	0,5 por item presente	**2,0**

Item	Critério de certo	Pontuação	Máx.
	gestão de tokens com tempos e commit no Git.		
P1 — Escolha do fluxo	Relaciona o fluxo ao tipo de cliente/presença de usuário **e** recomenda Authorization Code + PKCE (Client Credentials só servidor-servidor).	Certo: 1,0 · Sem justificativa: 0,5 · Recomenda Implicit/senha: 0	**1,0**
P2 — Passos de configuração	Cita na ordem ao menos 5 passos, incluindo registro, redirect URI exata e escopos mínimos.	≥5 passos: 1,0 · 3–4: 0,5 · <3: 0	**1,0**
P3 — Gestão de tokens	Explica expiração curta (janela de risco) + renovação por refresh + rotação ou revogação.	3 elementos: 1,0 · 2: 0,5 · ≤1: 0	**1,0**
P4 — Comunicação à equipe	Propõe ao menos 3 mecanismos concretos (documento/ADR, checklist de revisão, testes no CI, demonstração).	≥3: 1,0 · 2: 0,5 · ≤1: 0	**1,0**
Total			**10,0**

Respostas da atividade

Caso prático (roteiro 1.2)

1.2.1 — Escolha do fluxo

Authorization Code Flow com PKCE para o App Finanças: o usuário digita a senha apenas no provedor, o código trafega pelo navegador mas só vale junto com o `code_verifier` e o `client_secret`, que ficam no back-end. Para integrações internas sem usuário (relatórios, conciliação), **Client Credentials**. O Implicit Flow não é usado: ele entrega o token na URL e foi descontinuado pelas boas práticas atuais (RFC 9700 e OAuth 2.1), que recomendam Authorization Code + PKCE inclusive para SPAs.

1.2.2 — Configuração

- **Registro:** `client_id` `app-financas-web`, segredo em variável de ambiente/cofre, fluxos `authorization_code` e `refresh_token`.

- **Redirect URI:** somente `https://app.exemplo.com.br/callback` (no lab, `http://localhost:3000/callback`), comparação exata, sem curingas.
- **Escopos:** `saldo:read`, `transacoes:read` e, só com consentimento explícito, `transacoes:write`; o serviço interno recebe apenas `relatorios:read`.
- **Proteções:** `state` anti-CSRF, PKCE S256 obrigatório, código de uso único válido por 60 s.

1.2.3 — Gestão de tokens (exemplo)

Às 10h00 o usuário autoriza e recebe um access token JWT que expira às 10h05 e um refresh token opaco válido por 30 min. Às 10h06 a API responde 401; o back-end troca o refresh por um **novo par** e o refresh antigo é invalidado (**rotação**). Se o refresh antigo reaparecer, o provedor detecta o **reuso**, revoga toda a família e exige novo login. No logout, o app chama `/revoke`. No laboratório, isso foi observado no passo 15 com tokens de 20 s.

Perguntas da atividade

---

1. Como vocês explicariam a importância de escolher o fluxo OAuth adequado para nossa API e qual fluxo vocês recomendariam?

Cada fluxo foi desenhado para um tipo de cliente e um nível de confiança. A escolha define **onde o token trafega, quem consegue guardá-lo com segurança e se há um usuário consentindo**. Um fluxo errado abre brechas: o Implicit expõe o token na URL (histórico, logs, extensões); usar Client Credentials para dados de usuário elimina o consentimento; pedir a senha do usuário ao app anula o propósito do OAuth.

Recomendação: Authorization Code Flow com PKCE para o aplicativo que acessa dados dos clientes. O usuário se autentica só no provedor confiável, o código é de uso único e inútil sem o `code_verifier` e o `client_secret`, e o token é obtido em um canal servidor-servidor. Para integrações internas sem usuário, **Client Credentials**, com escopos próprios e restritos. No laboratório isso foi comprovado: o token do serviço de relatórios recebeu 403 ao tentar ler transações de usuários.

---

2. Quais são os passos fundamentais para configurar corretamente o OAuth, desde o registro da aplicação até a definição dos escopos?

1. **Registrar a aplicação** no provedor: `client_id`, `client_secret` (fora do código, em variável de ambiente ou cofre), tipo de cliente (confidencial/público) e fluxos permitidos.
2. **Cadastrar as URLs de redirecionamento** exatas, com HTTPS em produção e sem curingas; o provedor deve recusar qualquer outra.

3. **Definir escopos granulares** por recurso e operação (`saldo:read`, `transacoes:read`, `transacoes:write`) e pedir apenas o mínimo necessário (menor privilégio), com tela de consentimento clara.
4. **Montar o pedido de autorização** com `state` aleatório (anti-CSRF) e PKCE S256.
5. **Trocar o código por tokens no back-end** (`POST /token`) autenticando o cliente; código de uso único e curta validade.
6. **Configurar a API para validar o token**: assinatura, `exp`, `iss`, `aud` e escopo por rota, respondendo 401/403.
7. **Definir a política de tokens**: tempo de vida, refresh com rotação e revogação.
8. **Testar e monitorar**: testes automatizados das regras (como o `npm test` do lab), logs sem tokens e revisão periódica dos escopos.

---

3. De que maneira a gestão de tokens (expiração e renovação) contribui para a segurança da nossa API?

- **Expiração curta do access token** (5–15 min) limita a janela em que um token vazado pode ser usado: no lab, após 20 s a API respondeu 401 “token expirado”.
- **Renovação via refresh token** mantém a sessão sem pedir a senha de novo, então dá para ter tokens curtos sem prejudicar a experiência do usuário.
- **Rotação com detecção de reuso**: cada renovação gera um refresh novo e invalida o anterior; se o antigo reaparece (indício de roubo), toda a família é revogada — comportamento observado no passo 15.
- **Revogação** no logout ou em incidentes corta o acesso imediatamente no provedor.
- **Armazenamento seguro**: refresh opaco guardado só no back-end (nunca em localStorage), tokens fora de logs e URLs, sempre HTTPS.
- Os **escopos dentro do token** fazem com que, mesmo válido, ele só abra o que foi autorizado (403 na transferência).

---

4. Como vocês comunicariam essa estratégia para a equipe técnica, garantindo que todos estejam alinhados com as melhores práticas de OAuth?

- **Resposta objetiva no Slack** com a decisão e o link do plano (exemplo abaixo).
- **Documento de decisão** (plano/ADR no repositório) com diagrama de sequência do fluxo, tabela de escopos e política de tokens.
- **Checklist no code review** (modelo de PR): PKCE, `state`, redirect exata, escopos mínimos, segredos fora do código, validação `iss/aud/exp`.
- **Testes de segurança no CI**: o pipeline falha se uma regra for quebrada (como o `npm test` do laboratório).
- **Demonstração prática** em uma sessão curta, usando o ambiente do lab, e um canal único para dúvidas.
- **Componente compartilhado** (middleware de validação de token) para ninguém reimplementar a regra, e revisão do plano a cada novo escopo ou integração.

> “Time, proposta para a API financeira: Authorization Code + PKCE para o app, Client Credentials só para serviços internos; redirect URIs exatas, escopos mínimos (`saldo:read`, `transacoes:read`; `transacoes:write` só com consentimento); access token de 10 min e refresh com rotação. Plano e checklist no repositório (`docs/plano-oauth.md`); os testes já rodam no CI. Demo amanhã às 14h.”

Fontes

- SEDUC-SP — Roteiro de Atividade Prática “Descomplicando o OAuth: autorização segura para APIs” (SISANO2C2B4S25A1AP), Programação Back-End — material de referência
- RFC 6749 — The OAuth 2.0 Authorization Framework
  https://www.rfc-editor.org/rfc/rfc6749
- RFC 6750 — Bearer Token Usage
  https://www.rfc-editor.org/rfc/rfc6750
- RFC 7009 — OAuth 2.0 Token Revocation
  https://www.rfc-editor.org/rfc/rfc7009
- RFC 7636 — Proof Key for Code Exchange (PKCE)
  https://www.rfc-editor.org/rfc/rfc7636
- RFC 8414 — OAuth 2.0 Authorization Server Metadata
  https://www.rfc-editor.org/rfc/rfc8414
- RFC 9700 — Best Current Practice for OAuth 2.0 Security
  https://www.rfc-editor.org/rfc/rfc9700
- The OAuth 2.1 Authorization Framework (draft IETF)
  https://datatracker.ietf.org/doc/draft-ietf-oauth-v2-1/
- Node.js — documentação dos módulos crypto e http
  https://nodejs.org/api/crypto.html
- REST Client para VS Code
  https://marketplace.visualstudio.com/items?itemName=humao.rest-client

Feedback do aluno

O que funcionou, o que foi difícil e o que você mudaria nesta atividade?